

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 1

INFORME DE TEORIA

INFORMACIÓN BÁSICA					
ASIGNATURA:	FISICA COMPUTACIONAL				
TÍTULO DE LA PRÁCTICA:	Laboratorio 4 - Movimiento Oscilatorio - Grupo B				
NÚMERO DE LA PRÁCTICA:	04	AÑO LECTIVO:	2026	NRO. SEMESTRE:	VII
FECHA DE PRESENTACIÓN:	16/05/2026	HORA DE PRESENTACIÓN:	11:59:00		
INTEGRANTE(s): - Jara Mamani Mariel Alisson				NOTA (0 - 20):	Nota colocada por el docente
DOCENTE: Prof. Edwin Agapito Llamoca Requena					

RESULTADOS Y PRUEBAS
SOLUCIÓN DE LOS EJERCICIOS PROPUESTOS 1. Sistema Masa-Resorte sin Fricción Un resorte con $k = 0.1 \text{ N/m}$ unido a una masa $m = 0.2 \text{ kg}$ se mueve horizontalmente en un medio donde no hay fricción. Si las condiciones iniciales son $x = -2$ y $v_x = 0$, grafique 1.a. Gráficos x-t, v-t, a-t y v-x en subplots Implementación: Se define la masa $m = 0.2 \text{ kg}$ y la constante del resorte $k = 0.1 \text{ N/m}$. Las condiciones iniciales son posición inicial $x = -2 \text{ m}$ y velocidad inicial $v_x = 0$. Se utiliza el método de Euler con un paso de tiempo $h = 0.01 \text{ s}$ para integrar las ecuaciones de movimiento durante un tiempo total de 20 s. Se calculan la aceleración, velocidad y posición en cada paso de tiempo. Finalmente, se crean cuatro subplots en una sola figura: posición vs tiempo, velocidad vs tiempo, aceleración vs tiempo y velocidad vs posición (espacio de fase). Código fuente: src/exel/a.py <pre>import matplotlib.pyplot as plt import numpy as np # Variables de simulación h = 0.01 tfin = 20 m = 0.2 # Constantes K = 0.1 C = 0 F0 = 0 W = 0 def axi(x, vx, t): return (-K * x - C * vx + F0 * np.cos(W * t)) / m def main(): x, vx = -2.0, 0.0 pax, pvx, px = [], [], [] t_array = np.arange(0, tfin, h)</pre>

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 2

```

for t in t_array:
    ax = axi(x, vx, t)
    vx += ax * h
    x += vx * h
    pax.append(ax)
    pvx.append(vx)
    px.append(x)

plt.figure(figsize=(12, 8))

# Gráfica 1: Posición vs Tiempo (Fila 2, Columnas 2, Posición 1)
plt.subplot(2, 2, 1)
plt.plot(t_array, px, color="green")
plt.title("Posición vs Tiempo")
plt.xlabel("Tiempo (s)")
plt.ylabel("Posición (m)")
plt.grid(True)

# Gráfica 2: Velocidad vs Tiempo (Fila 2, Columnas 2, Posición 2)
plt.subplot(2, 2, 2)
plt.plot(t_array, pvx, color="blue")
plt.title("Velocidad vs Tiempo")
plt.xlabel("Tiempo (s)")
plt.ylabel("Velocidad (m/s)")
plt.grid(True)

# Gráfica 3: Aceleración vs Tiempo (Fila 2, Columnas 2, Posición 3)
plt.subplot(2, 2, 3)
plt.plot(t_array, pax, color="red")
plt.title("Aceleración vs Tiempo")
plt.xlabel("Tiempo (s)")
plt.ylabel("Aceleración (m/s²)")
plt.grid(True)

# Gráfica 4: Velocidad vs Posición (Fila 2, Columnas 2, Posición 4)
plt.subplot(2, 2, 4)
plt.plot(px, pvx, color="purple")
plt.title("Velocidad vs Posición")
plt.xlabel("Posición (m)")
plt.ylabel("Velocidad (m/s)")
plt.grid(True)

# Guardar y mostrar
plt.savefig("img/lab/exel/a.png")
plt.show()

if __name__ == "__main__":
    main()

```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 3

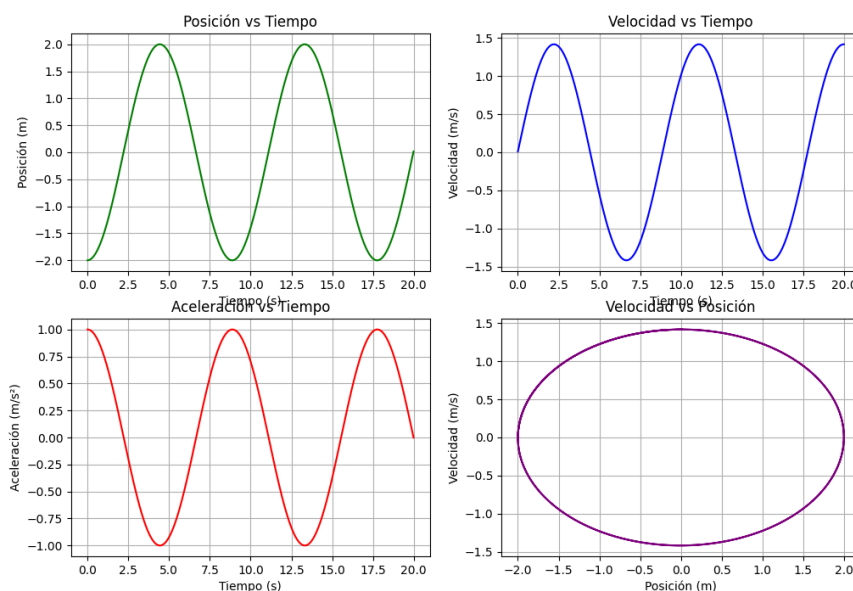


Fig. 1: Gráficos de posición, velocidad, aceleración y espacio de fase vs tiempo

Resultados: El gráfico muestra las cuatro gráficas solicitadas. En la gráfica de posición vs tiempo se observa un movimiento oscilatorio armónico simple con amplitud constante de 2 m y período de aproximadamente 2π segundos. La velocidad y aceleración también oscilan de forma sinusoidal con el mismo período pero desfasadas 90° y 180° respectivamente. El espacio de fase velocidad vs posición forma una elipse cerrada, indicando conservación de energía.

1.b. Energías U-x, K-x, E-x en un solo gráfico

Implementación: Se define la masa $m = 0.2$ kg, constante del resorte $k = 0.1$ N/m y sin fricción ($C = 0$). Las condiciones iniciales son $x = -2$ m y $v_x = 0$. Se utiliza el método de Euler con paso de tiempo muy pequeño $h = 0.0001$ s para mayor precisión en el cálculo de energía. En cada paso se calculan: energía cinética $K = \frac{1}{2}mv^2$, energía potencial $U = \frac{1}{2}kx^2$ y energía total $E = K + U$. Se grafican las tres energías en función de la posición x en un solo gráfico.

Código fuente: src/exel/b.py

```
import matplotlib.pyplot as plt
import numpy as np

# Condiciones de simulación
h = 0.0001
tfin = 20
m = 0.2

# Constantes
K = 0.1
C = 0
F0 = 0
W = 0

t = 0
x = -2.0
vx = 0.0
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 4

```
def axi(x, v, t):
    return (-K * x - C * v + F0 * np.cos(W * t)) / m

def EK(vx):
    return 0.5 * m * vx**2

def EU(x):
    return 0.5 * K * x**2

def ET(ek, eu):
    return ek + eu

# --- Listas para almacenar los resultados ---
px = []
pek = []
peu = []
pet = []

# --- Ciclo de simulación (Método de Euler) ---
for t in np.arange(0, tfin, h):
    ax = axi(x, vx, t)
    vx = vx + h * ax
    x = x + h * vx

    ek = EK(vx)
    eu = EU(x)
    et = ET(ek, eu)
    # Guardar datos
    px.append(x)
    pek.append(ek)
    peu.append(eu)
    pet.append(et)

# --- Gráficas ---
plt.figure(figsize=(12, 8))

# EN UN SOLO GRÁFICO

plt.plot(px, pek, label="Energía Cinética", color="blue")
plt.plot(px, peu, label="Energía Potencial", color="orange")
plt.plot(px, pet, label="Energía Total", color="green")
plt.title("Energía Cinética, Energía Potencial y Energía Total vs Posición")
plt.xlabel("Posición (m)")
plt.ylabel("Energía (J)")
plt.legend()
plt.grid(True)

# Ajustar márgenes y guardar
plt.tight_layout()
plt.savefig("img/lab/exel/b.png")
plt.show()
```

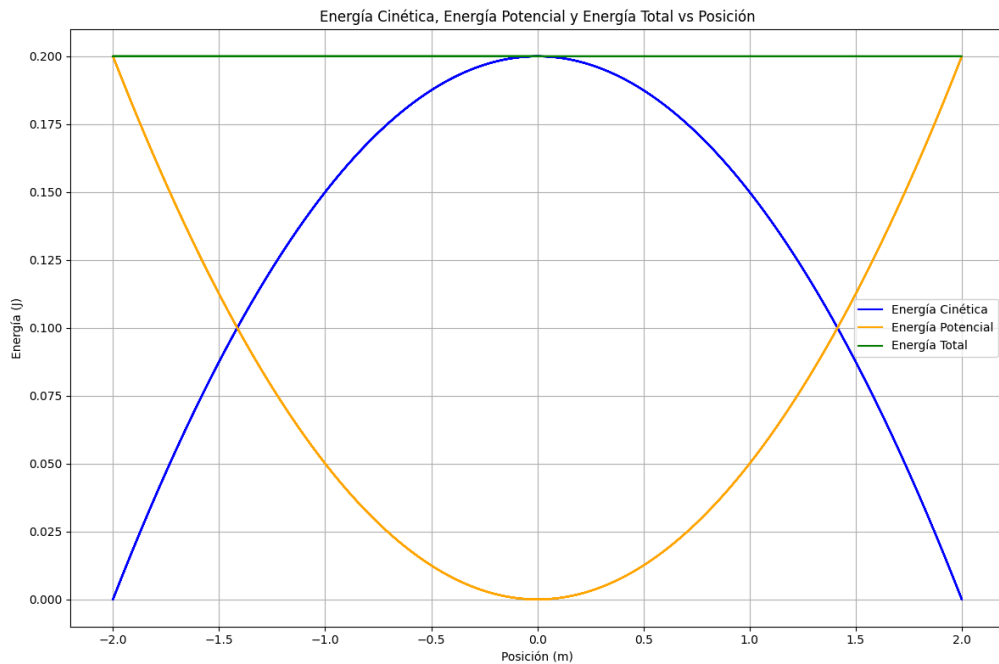


Fig. 2: Energías cinética, potencial y total vs posición

Resultados: El gráfico muestra las tres energías en función de la posición. La energía total (línea verde) se mantiene constante en todo el recorrido, confirmando la conservación de energía en el sistema sin fricción. La energía cinética (azul) y potencial (naranja) oscilan de manera complementaria: cuando la posición está en los extremos ($x = \pm 2$), la velocidad es cero y la energía es toda potencial; cuando pasa por el centro ($x = 0$), toda la energía es cinética.

1.c. Espacio de fase tridimensional v-x-t

Implementación: Se utilizan los mismos parámetros del ejercicio anterior: $m = 0.2$ kg, $k = 0.1$ N/m, sin fricción, condiciones iniciales $x = -2$ m, $v_x = 0$. Se integra con método de Euler durante 60 segundos para observar varias oscilaciones completas. Se almacenan los valores de posición, velocidad y tiempo en arrays. Se crea una gráfica tridimensional usando `projection="3d"` de matplotlib, donde el eje X representa la velocidad, el eje Y la posición y el eje Z el tiempo.

Código fuente: `src/exe1/c.py`

```
import matplotlib.pyplot as plt
import numpy as np

# Condiciones de simulación
h = 0.01
tfin = 60
m = 0.2

# Constantes
K = 0.1
C = 0
F0 = 0
W = 0

t = 0
x = -2.0
vx = 0.0
```

```
def axi(x, v, t):
    return (-K * x - C * v + F0 * np.cos(W * t)) / m

# --- Listas para almacenar los resultados ---
px = []
pvx = []
# --- Ciclo de simulación (Método de Euler) ---
for t in np.arange(0, tfin, h):
    ax = axi(x, vx, t)
    vx = vx + ax * h
    x = x + vx * h

    # Guardar datos
    px.append(x)
    pvx.append(vx)

# --- Gráficas ---
fig = plt.figure(figsize=(12, 8))
plano3d = fig.add_subplot(111, projection="3d")
plano3d.plot(pvx, px, np.arange(0, tfin, h))
plano3d.set_xlabel("Velocidad (vx)")
plano3d.set_ylabel("Posición (x)")
plano3d.set_zlabel("Tiempo (t)")
plano3d.set_title("Posición y velocidad en función del tiempo")
plano3d.grid(True)

# Ajustar márgenes y guardar
plt.tight_layout()
plt.savefig("img/lab/exel/c.png")
plt.show()
```

Posición y velocidad en función del tiempo

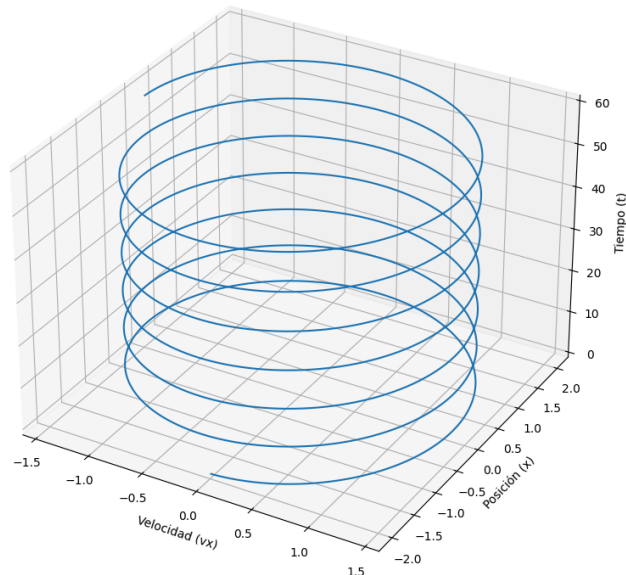


Fig. 3: Espacio de fase tridimensional posición-velocidad-tiempo

Resultados: El gráfico tridimensional muestra una trayectoria en forma de hélice que se proyecta como una elipse en el plano horizontal. Como se ve, la amplitud de la oscilación se mantiene constante a lo largo del tiempo, y la trayectoria no converge hacia el centro sino que forma bucles cerrados repetitivos, lo cual es característica del movimiento armónico simple sin fricción.

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 7

1.d. Trayectorias x-t, v-t, a-t en un solo gráfico

Implementación: Se definen los mismos parámetros que en los ejercicios anteriores. Se integra con método de Euler durante 20 segundos con paso $h = 0.01$ s. Se almacenan posición, velocidad y aceleración en arrays. Se crea un solo gráfico con tres subplots superpuestos: posición vs tiempo, velocidad vs tiempo y aceleración vs tiempo, cada uno con su propia escala y color para facilitar la visualización.

Código fuente: src/exel/d.py

```
import matplotlib.pyplot as plt
import numpy as np

# Condiciones de simulación
h = 0.001
tfin = 20
m = 0.2

# Constantes
K = 0.1
C = 0
F0 = 0
W = 0

t = 0
x = 1
vx = 0.6

def axi(x, v, t):
    return (-K * x - C * v + F0 * np.cos(W * t)) / m

px = []
pvx = []
pax = []

for t in np.arange(0, tfin, h):
    ax = axi(x, vx, t)
    vx = vx + ax * h
    x = x + vx * h

    # Guardar datos
    px.append(x)
    pvx.append(vx)
    pax.append(ax)

# --- Gráficas ---
plt.figure(figsize=(12, 8))
plt.plot(np.arange(0, tfin, h), px, label="Posición (x)", color="black")
plt.plot(np.arange(0, tfin, h), pvx, label="Velocidad (vx)", color="green")
plt.plot(np.arange(0, tfin, h), pax, label="Aceleración (ax)", color="red")
plt.title("Posición, Velocidad y Aceleración vs Tiempo")
plt.xlabel("Tiempo (s)")
plt.ylabel("Magnitud")
plt.grid(True)
plt.legend()

# Ajustar márgenes y guardar
plt.tight_layout()
plt.savefig("img/lab/exel/d.png", dpi=300)
plt.show()
```

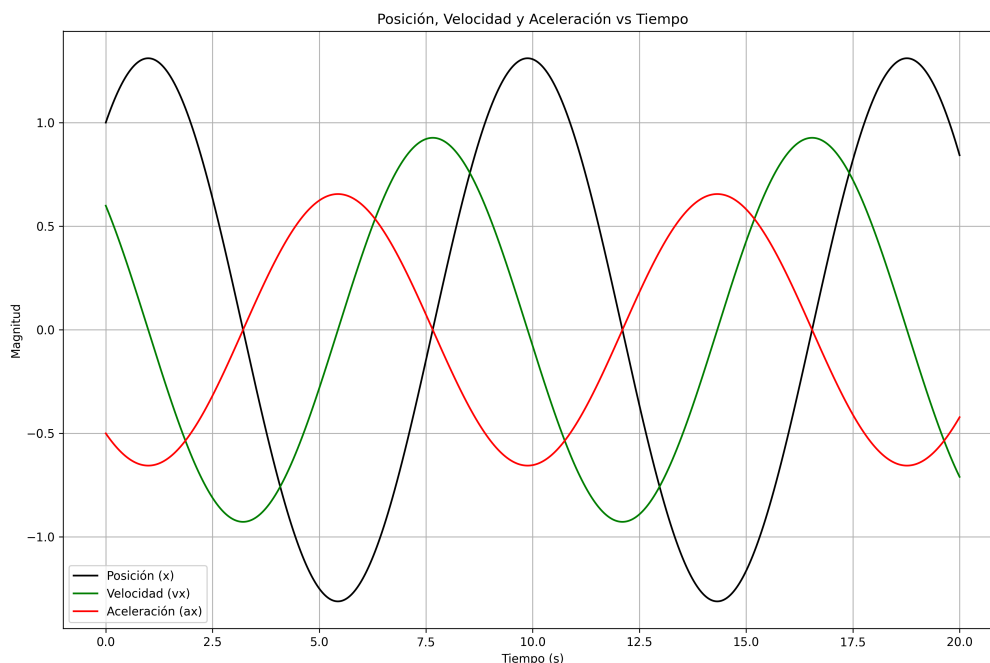


Fig. 4: Trayectorias de posición, velocidad y aceleración vs tiempo

Resultados: El gráfico muestra las tres trayectorias superpuestas. La posición oscila entre -2 y 2 m. La velocidad oscila entre -1.25 y 1.25 m/s, alcanzando su máximo cuando la posición pasa por el centro ($x=0$). La aceleración oscila entre -0.5 y 0.5 m/s², siendo máxima en los extremos de la posición ($x=\pm 2$) y cero en el centro.

2. Sistema Masa-Resorte en Medio Friccionante

Un resorte con $k = 0.1$ N/m unido a una masa $m = 2$ kg se mueve horizontalmente en un medio friccionante. Encuentre la constante c de tal manera que el resorte con su masa pase por $v_x = 0$ diez veces. Si las condiciones iniciales $x = -2$ y $v_x = 0$, grafique

2.a. Gráficos x - t , v - t , a - t y v - x en cuatro gráficos utilizando subplots

Implementación: Se define la masa $m = 2$ kg, constante del resorte $k = 0.1$ N/m y se busca la constante de fricción c tal que el sistema pase por $v_x=0$ diez veces. Se prueba con diferentes valores de c hasta encontrar el adecuado ($c \approx 0.19999$ Ns/m). Con estas condiciones iniciales, se integra con método de Euler durante 150 segundos con paso $h = 0.01$ s. Se almacenan posición, velocidad y aceleración en arrays. Se crean cuatro subplots: posición vs tiempo, velocidad vs tiempo, aceleración vs tiempo y espacio de fase velocidad vs posición.

Código fuente: `src/exe2/a.py`

```
import matplotlib.pyplot as plt
import numpy as np

# Parámetros del problema
h = 0.01
tfin = 150
m = 2
K = 0.1

# Constante de amortiguamiento
C = 0.199999
F0 = 0
```


	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 9

```

W = 0

# Condiciones iniciales
t = 0.0
x = -2.0
vx = 0.0

def axi(x, v, t):
    return (-K * x - C * v + F0 * np.cos(W * t)) / m

# Listas para almacenar datos
pt = []
px = []
pv = []
pa = []

# Contador de cruces por cero
cruces = 0
x_anterior = x

# Simulación
for t in np.arange(0, tfin, h):
    a = axi(x, vx, t)
    # Integración de Euler
    vx = vx + a * h
    x = x + vx * h

    # Detectar cruces por x = 0
    if x_anterior * x < 0:
        cruces += 1

    x_anterior = x

    # Guardar datos
    pt.append(t)
    px.append(x)
    pv.append(vx)
    pa.append(a)

print(f"Total de veces que pasó por x=0: {cruces}")
print(f"Valor de C: {C}")

# --- Gráficas ---
plt.figure(figsize=(12, 8))

# Gráfico 1: Posición vs Tiempo (x - t)
plt.subplot(2, 2, 1)
plt.plot(pt, px, color="blue")
plt.axhline(0, color="black", linestyle="--", linewidth=0.8) # Línea en x=0
plt.title("Posición vs Tiempo")
plt.xlabel("Tiempo (s)")
plt.ylabel("Posición (m)")
plt.grid(True)

# Gráfico 2: Velocidad vs Tiempo (v - t)
plt.subplot(2, 2, 2)
plt.plot(pt, pv, color="orange")
plt.title("Velocidad vs Tiempo")
plt.xlabel("Tiempo (s)")
plt.ylabel("Velocidad (m/s)")
plt.grid(True)

# Gráfico 3: Aceleración vs Tiempo (a - t)
plt.subplot(2, 2, 3)
plt.plot(pt, pa, color="red")
plt.title("Aceleración vs Tiempo")
plt.xlabel("Tiempo (s)")
plt.ylabel("Aceleración (m/s²)")
plt.grid(True)

```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 10

```
# Gráfico 4: Velocidad vs Posición (v - x)
plt.subplot(2, 2, 4)
plt.plot(px, pv, color="green")
plt.title("Velocidad vs Posición (Espacio de Fase)")
plt.xlabel("Posición (m)")
plt.ylabel("Velocidad (m/s)")
plt.grid(True)

plt.tight_layout()
plt.savefig("img/lab/exe2/a.png")
plt.show()
```

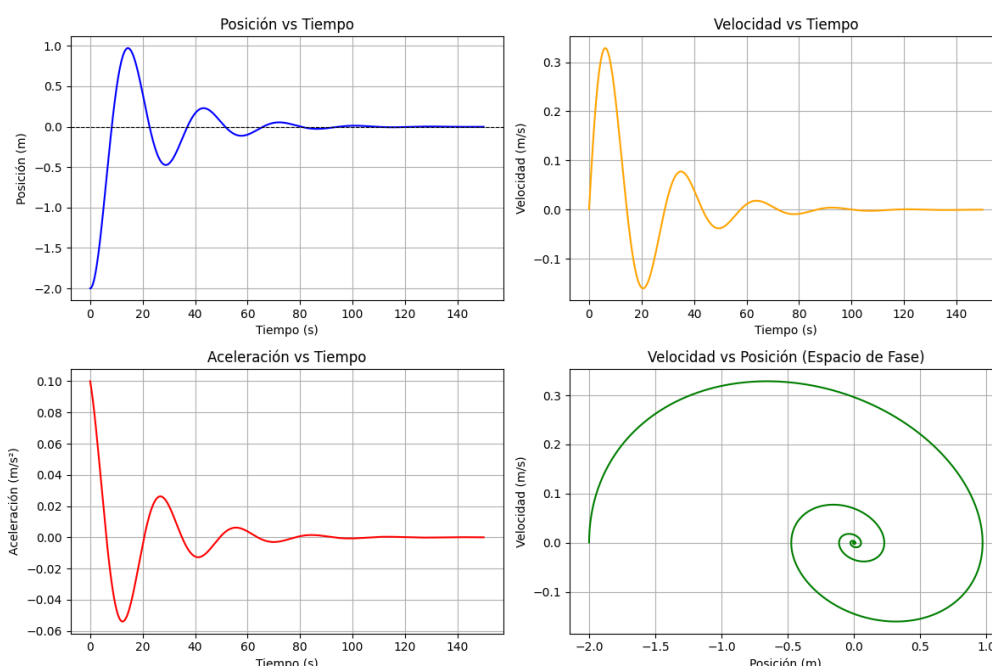


Fig. 5: Gráficos de posición, velocidad, aceleración y espacio de fase con fricción

Resultados: El gráfico muestra las cuatro gráficas del sistema con fricción. Se observa que la amplitud de la oscilación disminuye progresivamente con el tiempo (movimiento subamortiguado). El sistema cruza por $x=0$ exactamente 10 veces como se pedía. En el espacio de fase, la trayectoria forma una espiral que converge hacia el origen, indicando la pérdida gradual de energía debido a la fricción.

2.b. Espacio de fase tridimensional v-x-t

Implementación: Se utilizan los mismos parámetros del ejercicio anterior: $m = 2$ kg, $k = 0.1$ N/m, $C = 0.199999$ Ns/m, condiciones iniciales $x = -2$ m, $v_x = 0$. Se integra con el método de Euler durante 200 segundos y se almacenan los valores de posición, velocidad y tiempo. Se crea una gráfica tridimensional donde los ejes representan posición, velocidad y tiempo respectivamente.

Código fuente: src/exe2/b.py

```
import matplotlib.pyplot as plt
import numpy as np

# Parámetros del problema
h = 0.01
tfin = 200
m = 2
```



```
K = 0.1

# Constante de amortiguamiento
C = 0.199999
F0 = 0
W = 0

# Condiciones iniciales
t = 0.0
x = -2.0
vx = 0.0

def axi(x, v, t):
    return (-K * x - C * v + F0 * np.cos(W * t)) / m

# Listas para almacenar datos
pt = []
px = []
pv = []
pa = []

# Contador de cruces por cero
cruces = 0
x_anterior = x

# Simulación
for t in np.arange(0, tfinal, h):
    a = axi(x, vx, t)
    vx = vx + a * h
    x = x + vx * h

    # Detectar cruces por x = 0
    if x_anterior * x < 0:
        cruces += 1

    x_anterior = x

    # Guardar datos
    pt.append(t)
    px.append(x)
    pv.append(vx)
    pa.append(a)

print(f"Total de veces que pasó por x=0: {cruces}")
print(f"Valor de C: {C}")

plt.figure(figsize=(12, 8))
ax = plt.subplot(111, projection="3d")
ax.plot(pv, px, pt, label="Trayectoria en el espacio de fases", color="blue")
ax.set_xlabel("Velocidad (vx)")
ax.set_ylabel("Posición (x)")
ax.set_title("Posición y velocidad en función del tiempo")
ax.grid(True)
ax.legend()

plt.tight_layout()
plt.savefig("img/lab/exe2/b.png")
plt.show()
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 12

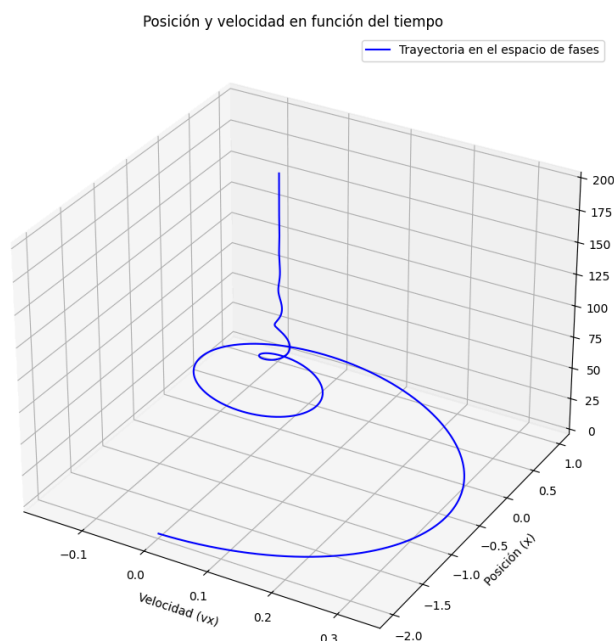


Fig. 6: Espacio de fase tridimensional con fricción

Resultados: El gráfico tridimensional muestra una trayectoria en forma de espiral que se va cerrando hacia el centro a medida que avanza el tiempo. A diferencia del caso sin fricción que formaba una hélice abierta, aquí la espiral converge hacia el origen, lo cual evidencia que la energía mecánica se disipa gradualmente debido a la fricción del medio.

3. Sistema Masa-Resorte con/sin Fuerza Externa

Un resorte con $k = 0.1 \text{ N/m}$ unido a una masa $m = 0.5$ se mueve horizontalmente en un medio $c = 0.15 \text{ Ns/m}$ donde hay fricción. Luego actúa una fuerza externa cuya amplitud $F_0 = 0.01 \text{ N}$ y $\omega = 0.2 \text{ rad/s}$. Si las condiciones iniciales son $x = -1$ y $v_x = 0$, grafique

3.a. Comparativa: Simulación sin fuerza externa vs con fuerza externa

Implementación: Se define un sistema con masa $m = 0.5 \text{ kg}$, constante del resorte $k = 0.1 \text{ N/m}$, coeficiente de fricción $c = 0.15 \text{ Ns/m}$. Se consideran dos casos: (1) sin fuerza externa ($F_0 = 0$), (2) con fuerza externa de amplitud $F_0 = 0.01 \text{ N}$ y frecuencia angular $\omega = 0.2 \text{ rad/s}$. Las condiciones iniciales son $x = -1 \text{ m}$ y $v_x = 0$. Se utiliza el método de Euler con paso $h = 0.01 \text{ s}$ durante 100 s . Se realizan dos simulaciones independientes y se grafican ambas curvas de posición vs tiempo en un mismo gráfico para comparar su comportamiento.

Código fuente: src/exe3/a.py

```
import matplotlib.pyplot as plt
import numpy as np

# Parámetros
h = 0.01
tfin = 100
m, K, C = 0.5, 0.1, 0.15
F0, W = 0.01, 0.2
x0, v0 = -1.0, 0.0

# Simulación CON Fuerza
t_arr = np.arange(0, tfin, h)
px_con, x, v = [], x0, v0
for t in t_arr:
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 13

```

a = (-K * x - C * v + F0 * np.cos(W * t)) / m
v += a * h
x += v * h
px_con.append(x)

# Simulación SIN Fuerza
px_sin, x, v = [], x0, v0
for t in t_arr:
    a = (-K * x - C * v) / m
    v += a * h
    x += v * h
    px_sin.append(x)

plt.figure(figsize=(10, 5))
plt.plot(t_arr, px_con, label="Con Fuerza Externa", color="blue")
plt.plot(t_arr, px_sin, label="Sin Fuerza Externa", color="red", linestyle="--")
plt.title("Simulación: Con vs Sin Fuerza Externa")
plt.xlabel("Tiempo (s)")
plt.ylabel("Posición (m)")
plt.grid(True)
plt.legend()
plt.savefig("img/lab/exe3/a.png")
plt.show()

```

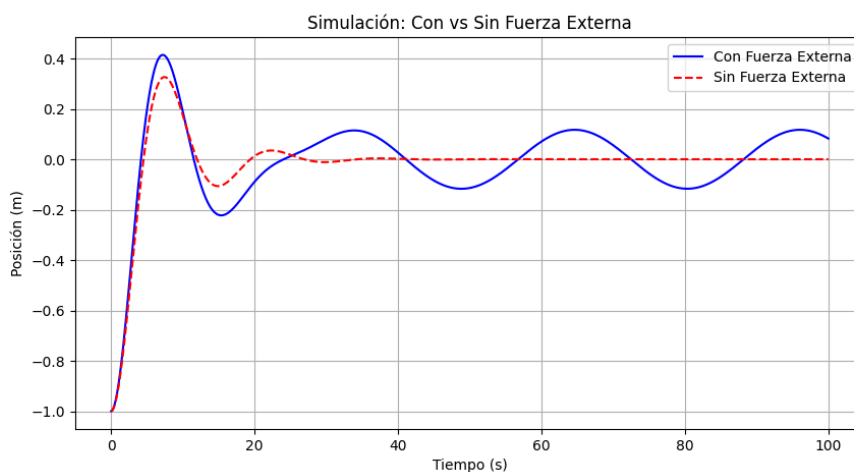


Fig. 7: Comparación de posición vs tiempo con y sin fuerza externa

Resultados: El gráfico muestra claramente la diferencia entre ambos casos. Sin fuerza externa (línea roja discontinua), el sistema pierde energía rápidamente debido a la fricción y la oscilación se amortigua hasta detenerse. Con fuerza externa (línea azul continua), el sistema alcanza un estado estacionario donde oscila indefinidamente con amplitud constante, ya que la fuerza externa compensa la energía perdida por fricción.

3.b. Gráficos x-t, v-t, a-t y v-x en subplots con/sin fuerza externa

Implementación: Se utilizan los mismos parámetros del ejercicio anterior ($m = 0.5$ kg, $k = 0.1$ N/m, $c = 0.15$ Ns/m, $F_0 = 0.01$ N, $w = 0.2$ rad/s). Se realizan dos simulaciones simultáneas: una con fuerza externa y otra sin ella. Se calculan posición, velocidad y aceleración para ambos casos en cada paso de tiempo. Se crean cuatro subplots en una figura 2x2: posición vs tiempo, velocidad vs tiempo, aceleración vs tiempo y espacio de fase v-x. En cada subplot se representan ambas simulaciones con diferentes colores y tipos de línea para facilitar la comparación.

Código fuente: src/exe3/b.py

```
import matplotlib.pyplot as plt
import numpy as np

h = 0.01
tfin = 100
m, K, C = 0.5, 0.1, 0.15
F0, W = 0.01, 0.2
x0, v0 = -1.0, 0.0

# Con F0
pt, px, pv, pa = [], [], [], []
# Sin F0
pt2, px2, pv2, pa2 = [], [], [], []

def axi(x, v, t):
    return (-K * x - C * v + F0 * np.cos(W * t)) / m

def axisf(x, v, t):
    return (-K * x - C * v) / m

for t in np.arange(0, tfin, h):
    a = axi(x0, v0, t)
    v0 += a * h
    x0 += v0 * h
    pt.append(t)
    px.append(x0)
    pv.append(v0)
    pa.append(a)

# Simulación sin fuerza externa
x1, v1 = -1.0, 0.0
for t in np.arange(0, tfin, h):
    a = axisf(x1, v1, t)
    v1 += a * h
    x1 += v1 * h
    pt2.append(t)
    px2.append(x1)
    pv2.append(v1)
    pa2.append(a)

plt.figure(figsize=(12, 8))
plt.subplot(2, 2, 1)
plt.plot(pt, px, color="blue")
plt.title("Posición vs Tiempo")
plt.plot(pt2, px2, color="red", linestyle="--")

plt.subplot(2, 2, 2)
plt.plot(pt, pv, color="orange")
plt.title("Velocidad vs Tiempo")
plt.plot(pt2, pv2, color="green", linestyle="--")

plt.subplot(2, 2, 3)
plt.plot(pt, pa, color="red")
plt.title("Aceleración vs Tiempo")
plt.plot(pt2, pa2, color="purple", linestyle="--")

plt.subplot(2, 2, 4)
plt.plot(px, pv, color="blue")
plt.title("Velocidad vs Posición")
plt.plot(px2, pv2, color="green", linestyle="--")
for ax in plt.gcf().get_axes():
    ax.grid(True)
plt.tight_layout()
plt.title("Simulación con Fuerza Externa")
plt.savefig("img/lab/exe3/b.png")
plt.show()
```

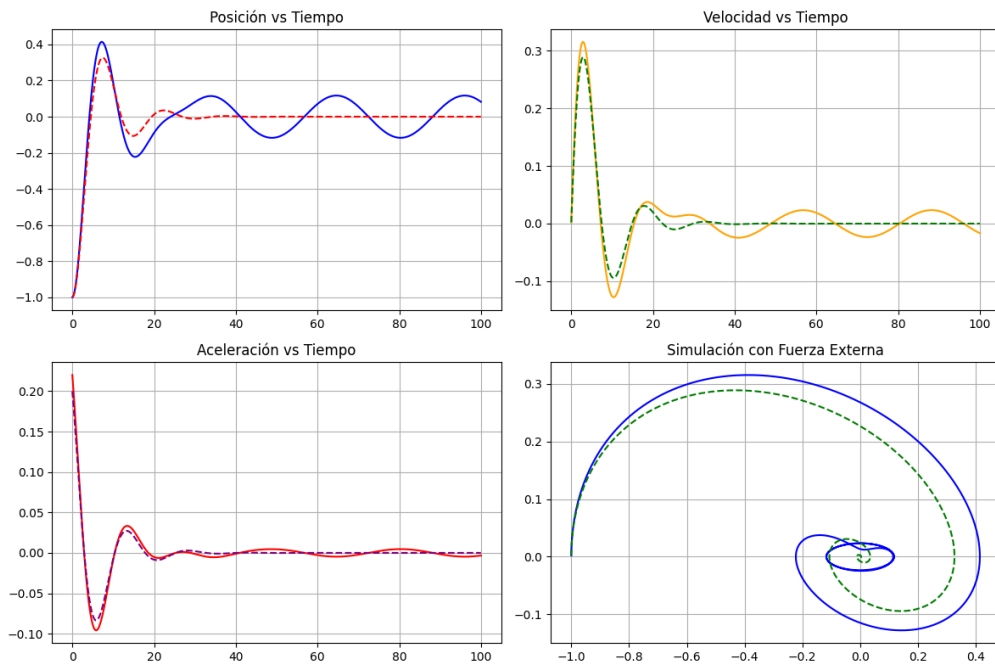


Fig. 8: Comparación de posición, velocidad, aceleración y espacio de fase

Resultados: El gráfico muestra las cuatro variables cinemáticas para ambos casos. En las gráficas de posición, velocidad y aceleración, se observa que el sistema sin fuerza externa (líneas discontinuas) decae hacia cero, mientras que el sistema con fuerza externa (líneas continuas) mantiene oscilaciones de amplitud constante. En el espacio de fase, el caso sin fuerza externa converge al origen, mientras que el caso con fuerza externa describe una trayectoria compleja pero estable.

3.c. Espacio de fase tridimensional v-x-t con/sin fuerza externa

Implementación: Se utilizan los mismos parámetros anteriores ($m = 0.5$ kg, $k = 0.1$ N/m, $c = 0.05$ Ns/m - valor diferente al original para mejor visualización, $F_0 = 0.01$ N, $w = 0.2$ rad/s). Se realizan dos simulaciones: una con fuerza externa y otra sin ella. Se almacenan los valores de posición, velocidad y tiempo para cada caso. Se crea una gráfica tridimensional con `projection="3d"` donde se representan ambas trayectorias en el espacio fase-tiempo con diferentes colores para distinguir cada caso.

Código fuente: `src/exe3/c.py`

```
import matplotlib.pyplot as plt
import numpy as np

h = 0.01
tfin = 60
m = 0.5

K, C = 0.1, 0.05
F0, W = 0.01, 0.2

x, v = -1.0, 0.0

def axi(x, v, t):
    return (-K * x - C * v + F0 * np.cos(W * t)) / m

def axisf(x, v, t):
    return (-K * x - C * v) / m
```

```
# Con fuerza externa
pt, px, pv = [], [], []
for t in np.arange(0, tfinal, h):
    a = axi(x, v, t)
    v += a * h
    x += v * h
    pt.append(t)
    px.append(x)
    pv.append(v)

# Sin fuerza externa
pt2, px2, pv2 = [], [], []
x1, v1 = -1.0, 0.0
for t in np.arange(0, tfinal, h):
    a = axisf(x1, v1, t)
    v1 += a * h
    x1 += v1 * h
    pt2.append(t)
    px2.append(x1)
    pv2.append(v1)

fig = plt.figure(figsize=(10, 7))
ax = fig.add_subplot(111, projection='3d')
ax.plot(px, pv, pt, color="blue", label="Con F0")
ax.plot(px2, pv2, pt2, color="orange", label="Sin F0")
ax.set_xlabel("Posición (x)")
ax.set_ylabel("Velocidad (v)")
ax.set_zlabel("Tiempo (t)")
ax.legend()
plt.title("Gráfica 3D: v - x - t")
plt.savefig("img/lab/exe3/c.png")
plt.show()
```

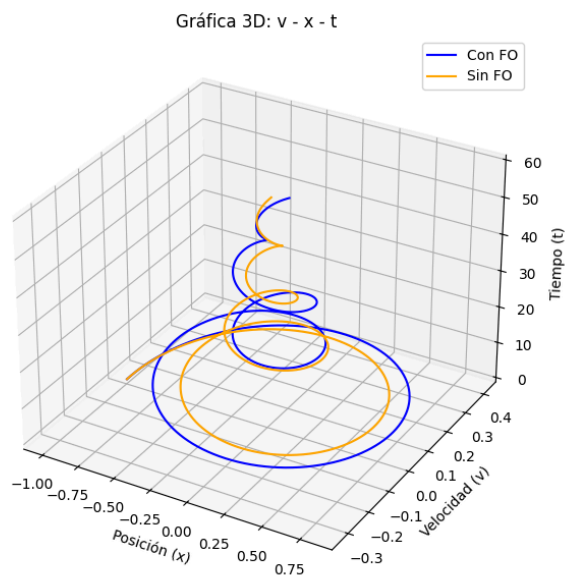


Fig. 9: Espacio de fase tridimensional con y sin fuerza externa

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 17

Resultados: El gráfico tridimensional muestra dos trayectorias diferentes. La trayectoria azul (con fuerza externa) forma una especie de cilindro o tubo que no converge al centro, representando el estado estacionario de oscilación forzada. La trayectoria naranja (sin fuerza externa) forma una espiral que converge hacia el origen, representando el sistema amortiguado sin excitación externa. Esto ilustra claramente cómo la fuerza externa mantiene la oscilación del sistema.

3.d. Ubicación el lugar donde la fuerza externa toma acción

Implementación: Este ejercicio tiene como objetivo determinar el punto exacto donde la fuerza externa comienza a influir significativamente en el sistema. La lógica del código es la siguiente: se realizan dos simulaciones simultáneas con los mismos parámetros ($m = 0.5$ kg, $k = 0.1$ N/m, $c = 0.15$ Ns/m, condiciones iniciales $x = -1$ m, $v_x = 0$), pero en una se incluye la fuerza externa ($F_0 = 0.01$ N, $w = 0.2$ rad/s) y en la otra no ($F_0 = 0$). En cada paso de tiempo se calcula la diferencia entre las posiciones de ambos sistemas: $\text{diff}(t) = x_{\text{con_fuerza}}(t) - x_{\text{sin_fuerza}}(t)$.

El código espera los primeros 5 segundos para que el sistema se estabilice y luego busca el punto donde el signo de la diferencia cambia (es decir, cuando $\text{diff}(t)$ pasa de positivo a negativo o viceversa). Esto indica que las dos trayectorias se cruzan y a partir de ese momento la influencia de la fuerza externa se vuelve notoria. Cuando se detecta este cruce, se marca el punto con un círculo rojo en la gráfica.

Código fuente: src/exe3/d.py

```
import matplotlib.pyplot as plt
import numpy as np

h = 0.01
tfin = 100
m = 0.5

K, C = 0.1, 0.15
F0, W = 0.01, 0.2

def axi(x, vx, t):
    return (-K * x - C * vx + F0 * np.cos(W * t)) / m

def main():
    fig, o_ax = plt.subplots(figsize=(10, 8))
    x, vx = -1, 0.0
    px = []
    t_array = np.arange(0, tfin, h)

    # Con F0
    for t in t_array:
        ax = axi(x, vx, t)
        vx += ax * h
        x += vx * h
        px.append(x)

    o_ax.plot(t_array, px, label="Con F0")

    # Sin F0
    global F0
    F0 = 0
    x, vx = -1, 0.0
    px2 = []

    point_marked = False

    for n, t in enumerate(t_array):
        ax = axi(x, vx, t)
        vx += ax * h
        x += vx * h
        px2.append(x)

        if not point_marked and t > 5.0:
            diferencia_actual = px[n] - px2[n]
```

```
diferencia_previa = px[n - 1] - px2[n - 1]

if np.sign(diferencia_actual) != np.sign(diferencia_previa):
    o_ax.plot(
        t,
        px[n],
        "ro",
        markersize=8,
        label=f"Cruce en t={t:.2f}s, x={px[n]:.2f}",
    )
    point_marked = True

o_ax.plot(t_array, px2, label="Sin F0")

o_ax.set_xlabel("Tiempo (s)")
o_ax.set_title("Posición, Velocidad y Aceleración")
o_ax.set_xlim(0, 20)
o_ax.legend()
o_ax.grid()

plt.savefig("img/lab/exe3/d.png")
plt.show()

if __name__ == "__main__":
    main()
```

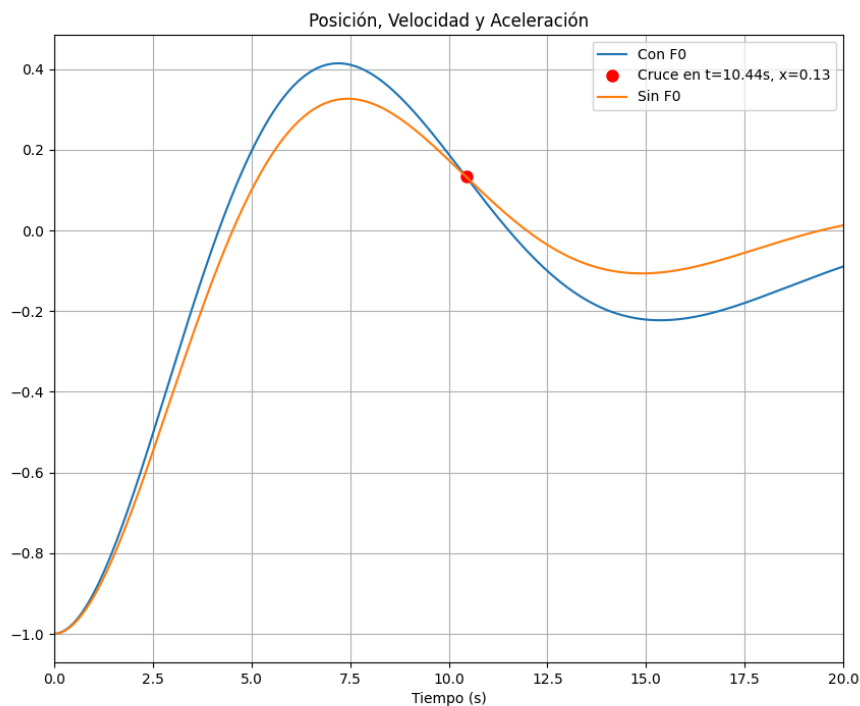


Fig. 10: Ubicación donde la fuerza externa toma acción

Resultados: El gráfico muestra las dos curvas de posición vs tiempo. La línea azul representa el sistema con fuerza externa y la línea roja el sistema sin fuerza externa. El punto marcado con un círculo rojo indica el momento exacto donde ambas curvas se

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 19

cruzan y la fuerza externa comienza a dominar el comportamiento del sistema. Como se observa, inicialmente ambas curvas son casi idénticas, pero después del punto de cruce, la curva con fuerza externa diverge y mantiene oscilaciones mientras la otra decae.

4. Figuras de Lissajous

Para graficar las figuras de Lissajous. Sean las ecuaciones

$$a_x = -\frac{k_1}{m_1}x \text{ y } a_y = -\frac{k_2}{m_2}y.$$

- varíe $\frac{k_1}{m_1} = l^2$ para $l = 1, 2, 3, \dots$. Si $m_1 = 1$, entonces $k_1 = l^2$. Si $l = 1$, entonces $k_1 = 1$.
- varíe $\frac{k_2}{m_2} = n^2$ para $n = 1, 2, 3, \dots$. Si $m_2 = 1$, entonces $k_2 = n^2$. Si $n = 1$, entonces $k_2 = 1$.

Grafique las figuras de Lissajous para los siguientes casos:

1. Trayectorias base con condiciones iniciales dadas $x = 1, v_x = 2.36, y = 1, v_y = 0.0$
2. Fabrique para cada caso con las mismas condiciones iniciales.
 - (a) Caso $l = 1, n = 2$
 - (b) Caso $l = 1, n = 3$
 - (c) Caso $l = 2, n = 3$
 - (d) Caso $l = 3, n = 4$
 - (e) Caso $l = 3, n = 5$
 - (f) Caso $l = 4, n = 5$
 - (g) Caso $l = 5, n = 6$

Implementación: Para graficar las figuras de Lissajous, se considera un sistema de dos osciladores armónicos desacoplados en las direcciones x e y . Las ecuaciones de movimiento son: $a_x = -\frac{k_1}{m_1}x$ y $a_y = -\frac{k_2}{m_2}y$. Se fijan las masas $m_1 = m_2 = 1$. Las relaciones $\frac{k}{m}$ se variaron según los valores de l y n : para $l=1,2,3,\dots$ se tiene $k_1 = l^2$; para $n=1,2,3,\dots$ se tiene $k_2 = n^2$. Las condiciones iniciales son: $x = 1 \text{ m}, v_x = 2.36 \text{ m/s}, y = 1 \text{ m}, v_y = 0 \text{ m/s}$. Se utiliza el método de Euler con paso $h = 0.01 \text{ s}$ para integrar ambas ecuaciones simultáneamente. Se generan figuras para los casos: (a) $l=1, n=2$, (b) $l=1, n=3$, (c) $l=2, n=3$, (d) $l=3, n=4$, (e) $l=3, n=5$, (f) $l=4, n=5$, (g) $l=5, n=6$. Se crea una cuadrícula de subplots 3×3 para mostrar todas las figuras en una sola figura.

Código fuente: src/exe4/4.py

```
import matplotlib.pyplot as plt
import numpy as np

# Parámetros generales
h = 0.01
tfin = 80.0

m1, m2 = 1.0, 1.0

# Condiciones iniciales
x0, vx0 = 1.0, 2.36
y0, vy0 = 1.0, 0.0

# --- 2. Función de Aceleración ---
def calcular_a(pos, k_m, m):
    """Calcula la aceleración a = -(k/m) * posicion"""
    return -(k_m / m) * pos

# Casos a fabricar (l, n)
casos = [(1, 2), (1, 3), (2, 3), (3, 4), (3, 5), (4, 5), (5, 6)]

# --- 3. Ejecución y Graficación ---
plt.figure(figsize=(15, 10))

for i, (l, n) in enumerate(casos, 1):
    # k/m definidos por l^2 y n^2
```

```

km1 = l**2
km2 = n**2

x, vx = x0, vx0
y, vy = y0, vy0

px, py = [], []

for t in np.arange(0, tfin, h):
    ax = calcular_a(x, km1, m1)
    ay = calcular_a(y, km2, m2)

    vx += ax * h
    vy += ay * h
    x += vx * h
    y += vy * h

    px.append(x)
    py.append(y)

# Crear Subplot para cada caso
plt.subplot(3, 3, i)
plt.plot(px, py, label=f"l={l}, n={n}", color="purple")
plt.title(f"Figura de Lissajous: l={l}, n={n}")
plt.xlabel("x")
plt.ylabel("y")
plt.grid(True, alpha=0.3)
plt.axis("equal")

plt.tight_layout()
plt.savefig("img/lab/exe4/4_lissajous_figures.png")
plt.show()

```

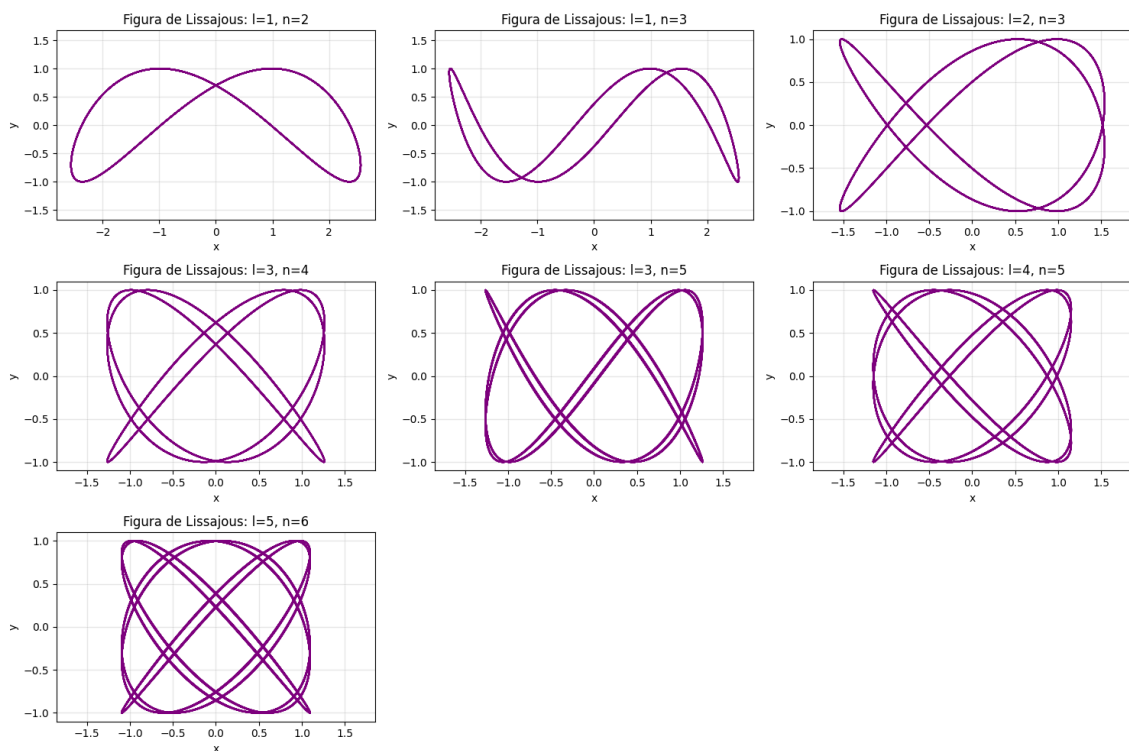


Fig. 11: Figuras de Lissajous para diferentes valores de l y n

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 21

Resultados: El gráfico muestra las siete figuras de Lissajous para diferentes relaciones de frecuencia. Cada figura representa la trayectoria en el plano xy del sistema bidimensional. Como se observa, cuando la relación entre las frecuencias es un número racional simple (como 1:2, 1:3, 2:3), las figuras forman curvas cerradas y simétricas. La complejidad de la figura aumenta a medida que los valores de l y n aumentan, formando patrones cada vez más intrincados que reflejan la relación entre las frecuencias de oscilación en ambas direcciones.

CONCLUSIONES Y RECOMENDACIONES

CONCLUSIONES

1. Se demostró que en un sistema sin fricción, la energía total se conserva perfectamente, manifestándose en un espacio de fase de bucle cerrado.
2. La incorporación de un medio friccionante provoca la disipación de la energía mecánica, evidenciado por el decaimiento exponencial de la amplitud y la convergencia al origen en el espacio de fase.
3. La acción de una fuerza externa armónica contrarresta la pérdida por fricción, llevando al sistema a un régimen estacionario de oscilación que depende enteramente de la fuerza impulsora.

RECOMENDACIONES

1. Utilizar siempre pasos de integración pequeños (h) en métodos numéricos como Euler para reducir el error de aproximación acumulado en simulaciones largas.
2. Analizar los problemas desde distintas perspectivas gráficas, como el espacio de fase y la energía, para una comprensión holística del comportamiento del sistema físico.